

Data Analysis with Python

Lesson 1: Introduction to Python and Jupyter

Introduction to Python and Jupyter - Study Notes

Key Concepts

1. **Python Basics** – Syntax, variables, data types, and basic I/O.
2. **Environment Setup** – Installing Python, using package managers (pip, conda), and configuring virtual environments.
3. **Jupyter Notebooks** – Web based interactive notebooks that combine code, visualizations, and narrative text.
4. **Cell Execution & Magic Commands** – Running code cells, re running, clearing outputs, and using `%`` and `%%`` magic for quick tasks.
5. **Exporting & Sharing** – Converting notebooks to HTML, PDF, or Python scripts for collaboration.

Summary

Python is a high level, interpreted language known for its readability and vast ecosystem of libraries. Setting up a clean environment—whether via `venv``, `virtualenv``, or Conda—ensures reproducibility and avoids dependency clashes. Jupyter Notebooks provide an interactive workspace where you can write code, instantly see results, and embed Markdown explanations, making them ideal for learning, data exploration, and prototyping.

A typical workflow starts with installing Python, creating a virtual environment, installing Jupyter (`pip install jupyter`` or `conda install jupyter``), and launching the notebook server with `jupyter notebook``. Inside the notebook, you can mix code cells (Python) with Markdown cells to document your thought process. Magic commands (e.g., `%timeit``, `%%capture``) give you quick access to useful tools without leaving the notebook interface. Finally, notebooks can be exported to share with others or converted into scripts for production use.

Important Points

- **Virtual Environments**: Always isolate project dependencies (`python -m venv env`` or `conda create -n myenv python=3.12``).
- **Jupyter Launch**: `jupyter notebook`` opens a local web server; `jupyter lab`` offers a more modern interface.
- **Cell Types**: Code cells run Python; Markdown cells support headings, lists, images, and LaTeX.
- **Execution Order**: Cells can be run out of order; outputs are tied to the cell that produced them.
- **Magic Commands**: Single line (`%``) and cell level (`%%``) commands extend functionality (e.g., `%matplotlib inline`` for plots).
- **Exporting**: Use `File !`` Download as` to export notebooks to HTML, PDF, or Python script.
- **Version Control**: Store notebooks in Git; consider using `nbdime`` for diffing notebook changes.

Examples

1. Hello World & Simple Math

```
```python
```

#### Cell 1: Basic output

```
print("Hello, Python!")
```

#### Cell 2: Arithmetic

```
a = 10
```

```
b = 3
```

```
print(f'{a} + {b} = {a + b}')
```

```
```
```

Explanation: The first cell prints a greeting. The second cell demonstrates variable assignment and f string formatting.

2. Plotting with Matplotlib in a Notebook

```
```python
```

#### Cell 1: Import libraries

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

#### Cell 2: Create data

```
x = np.linspace(0, 2*np.pi, 100)
```

```
y = np.sin(x)
```

#### Cell 3: Plot

```
plt.figure(figsize=(6,4))
```

```
plt.plot(x, y, label='sin(x)')
```

```
plt.title('Sine Wave')
```

```
plt.xlabel('x')
```

```
plt.ylabel('sin(x)')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
```
```

Explanation: The `%matplotlib inline`` magic (enabled by default in Jupyter) renders the plot directly below the cell. This example shows how to generate and display a simple line plot.

```
---
```

Quick Review

1. **What command creates a new virtual environment in Python 3?**
2. **How do you launch the Jupyter Notebook server from the terminal?**
3. **Name two types of cells available in a Jupyter notebook.**
4. **What magic command would you use to time the execution of a single line of code?**
5. **Explain one advantage of using Jupyter notebooks for data exploration.**

Further Reading

- **Python Standard Library** – ``datetime``, ``collections``, ``os``, ``json``.
- **Package Management** – ``pip``, ``conda``, ``poetry``, and ``pipenv``.
- **Data Science Libraries** – ``pandas``, ``NumPy``, ``scikit-learn``.
- **Advanced Jupyter Features** – Widgets (``ipywidgets``), JupyterLab extensions, and JupyterHub.
- **Version Control with Notebooks** – ``nbdime``, Git best practices for ``ipynb`` files.
- **Testing Python Code** – ``unittest``, ``pytest``, and test-driven development.

